pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Analyzing Text

WEEK 4 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Text is data too

Today's text: FOMC statements, one row for each statement

46

statements, one row each

2021-01-27

date of the first statement

2026-09-16

date of the last statement

The three columns of the file

Column	What it holds
`date`	the date of the statement, as text
`url`	the address of the statement's page
`text`	the statement as published, paragraphs separated by a line break

```
statements = pd.read_csv(data_file)
print(statements.shape)   # (46, 3)
```

The portfolio manager asks for a table

1. How long is each statement?
2. Which figures does it contain?
3. How often do a few chosen words appear?
4. How much wording does it share with the last?

Today we count words. We do not say what a statement means, and no tone is scored.

Reuse basis, quoted from the Board's website

https://www.federalreserve.gov/disclaimer.htm

"Unless otherwise indicated, information on Board's website is in the public domain and may be copied and distributed without permission. Please cite to the Board as the source of the information."

Source: Board of Governors of the Federal Reserve System, Federal Open Market Committee statements, federalreserve.gov.

Calendar page: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm. Retrieved on 2026-10-03. Text exactly as published.

Strings again: ten tools, ten Excel twins

Statement dated 2023-11-01, first paragraph: three sentences, stored as opening. For the Excel formulas it is in cell A2.

> Recent indicators suggest that economic activity expanded at a strong pace in the third quarter.
> Job gains have moderated since earlier in the year but remain strong, and the unemployment rate has remained low.
> Inflation remains elevated.

Job	Python	Excel	Result
Number of characters	`len(opening)`	`=LEN(A2)`	238
Lower case	`opening.lower()`	`=LOWER(A2)`	a new string
Capitals	`opening.upper()`	`=UPPER(A2)`	a new string
Is it there	`'inflation' in opening`	`=ISNUMBER(FIND("inflation", A2))`	True or False
Where it starts	`opening.find('Inflation')`	`=FIND("Inflation", A2)`	211 (Excel: 212)
A slice	`opening[:6]`	`=LEFT(A2, 6)`	Recent
How many times	`opening.count('rate')`	`=(LEN(A2) - LEN(SUBSTITUTE(A2, "rate", ""))) / LEN("rate")`	2
Cut into a list	`opening.split()`	`=TEXTSPLIT(A2, " ")`	37 words
Glue a list together	`' '.join(words[:5])`	`=TEXTJOIN(" ", TRUE, B2:F2)`	one string
Spaces off the ends	`piece.strip()`	`=TRIM(B2)`	no end spaces
Swap text	`opening.replace('.', '')`	`=SUBSTITUTE(A2, ".", "")`	3 full stops gone

Python counts positions from 0 and Excel from 1: find() returns 211 and FIND returns 212. TEXTSPLIT is in Microsoft 365 and Excel 2024.

A regular expression, piece by piece

```
range_pattern = r'[\d/-]+ to [\d/-]+ percent'
```

`[\d/-]+`	`to`	`[\d/-]+`	`percent`
the first figure: digits, slashes, and hyphens	a space, the letters t and o, a space	the second figure	a space and the word
matched `5-1/4`	matched `to`	matched `5-1/2`	matched `percent`

Inside the first piece

Piece	Meaning
`[ ]`	one character from the list inside
`\d`	any one digit
`/`	a slash
`-`	a plain hyphen, placed last
`+`	one or more of what is before it

Statement dated 2023-11-01, third paragraph, second sentence, as published:

In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.

```
re.findall(range_pattern, policy)
# ['5-1/4 to 5-1/2 percent']
match = re.search(r'([\d/-]+) to ([\d/-]+) percent', policy)
match.group(1), match.group(2), match.start()   # 5-1/4, 5-1/2, 218
re.sub(range_pattern, '[RANGE]', sentence)   # ... rate at [RANGE].
```

The same in Excel for Microsoft 365

```
=REGEXEXTRACT(A2, "[\d/-]+ to [\d/-]+ percent")
```

returns 5-1/4 to 5-1/2 percent

The sentence is in A2. Not in earlier versions.

Regular expressions: the pieces

Piece	Meaning	A pattern from the notebook	What that pattern describes
`\d`	any one digit	`\d+ percent`	digits, a space, and the word percent
`\w`	one word character: a letter, a digit, or an underscore	`^\w+`	the first word of the text
`\s`	one white space character: a space, a tab, or a line break	`\s+`	a run of white space
`\b`	the edge of a word: a position, not a character	`\brate\b`	rate as a whole word
`+`	one or more of whatever is just before it	`[\d/-]+`	a figure such as 5-1/4
`*`	none or more of whatever is just before it	`\w*rate\w*`	rate with any letters attached
`?`	none or one, which makes a piece optional	`(?:[A-Z]\. )?`	a middle initial, or none
`{4}`	exactly 4 of whatever is just before it	`^\d{4}-\d{2}-\d{2}$`	the shape of a date: 2023-11-01
`^ $`	anchors: the start and the end of the text or of a line	`\w+\.$`	a word and a full stop, ending a line
`( )`	a group: a part of the match, handed back by itself	`([\d/-]+) to ([\d/-]+) percent`	the two ends: 5-1/4 and 5-1/2
`[ ]`	one character from a list. [^] is one not on the list	`[^a-z\s]`	not a lower-case letter or white space
`.`	any one character. \. is a real full stop	`.{12}`	any twelve characters

Write every pattern as r'...', a raw string: a backslash is only a backslash. re.search() returns None when nothing matches.

Excel for Microsoft 365: `=REGEXTEST(A2, pattern)` `=REGEXEXTRACT(A2, pattern)` `=REGEXREPLACE(A2, pattern, new)` test, pull out, replace

Every version of Excel: wildcards in COUNTIF, SUMIF, SEARCH and the Find box. * is any run of characters, ? is any one. Flash Fill: Ctrl+E.

Too much, too little, and the fix

```
re.findall(r'\d+ percent', policy)
# ['2 percent', '2 percent', '2 percent', '2 percent']
re.findall(r'[\d/-]+ percent', policy)
# ['2 percent', '5-1/2 percent', '2 percent', '2 percent']
```

Matched too much, then fixed

4 matches, and the paragraph has 2 percent 3 times.

The fourth is the tail of 5-1/2 percent.

With [\d/-]+ each figure is whole: 3 read 2 percent.

```
statement.count('rate')   # 4
re.findall(r'\w*rate\w*', statement)
# ['moderated', 'rate', 'rate', 'rate']
re.findall(r'\brate\b', statement)
# ['rate', 'rate', 'rate']
```

Whole words: \b is the edge of a word

count() finds the four letters, inside moderated too.

With \b on each side, the word rate appears 3 times

in the statement dated 2023-11-01.

```
voting.count(';') + 1   # 12 names, counted by semicolons
re.findall(r'[A-Z][a-z]+ [A-Z]\. [A-Z][a-z]+', voting)   # 10
re.findall(r'[A-Z][a-z]+ (?:[A-Z]\. )?[A-Z][a-z]+', voting)   # 13
```

Matched too little, then too much

A middle initial required: 10 of the 12 names match.

Initial optional: 13. One match is a title, not a name.

Without the title: 12, the count by semicolons.

Print what a pattern matches before trusting how many. Test it against a count from somewhere else.

In Excel `=COUNTIF(B2:B38, "*rate*")` returns 2: moderated and rate. `=COUNTIF(B2:B38, "rate")` returns 1. The 37 words are in B2:B38.

Patterns for a desk: descriptions and CUSIPs

```
description_pattern = r'([A-Z]+)\s+(\d+(?:\.\d+)?)\s+(\d{2}/\d{2}/\d{4})'
```

Below each piece: its match in KRIX 7.875 08/26/2032

`([A-Z]+)`	`\s+`	`(\d+(?:\.\d+)?)`	`\s+`	`(\d{2}/\d{2}/\d{4})`
group 1, the ticker: one or more capital letters `KRIX`	one or more spaces `a space`	group 2, the coupon: digits, and an optional decimal part `7.875`	one or more spaces `a space`	group 3, the maturity: two digits, a slash, two digits, a slash, four digits `08/26/2032`

```
first_pattern = r'([A-Z]+) (\d+\.\d+) (\d{2}/\d{2}/\d{4})'
# Read: 6 of 8
# Not read: ['BEZE 7 06/18/2032', 'OSTV  6.5 04/01/2033']
assert match is not None, f'cannot read this description: {text}'
# Read: 8 of 8   mean coupon: 6.609 percent   latest maturity: 2055-09-03
```

Matched too little: 6 of 8, then 8 of 8

BEZE 7 has a coupon with no decimal point.

The OSTV line has two spaces after the ticker.

With \s+ and the optional (?:\.\d+)? all 8 are read.

Job	Python	Result in the notebook	Excel
Greedy	`re.search(r' at (.+) percent', policy)`	858 of the 944 characters	
Lazy	`re.search(r' at (.+?) percent', policy)`	13 characters: the rate of 2	`=TEXTBEFORE(TEXTAFTER(A2, " at "), " percent")`
A whole value	`re.fullmatch(r'\d{4}-\d{2}-\d{2}', value)`	False for a date plus a line break	`=REGEXTEST(A2, "^\d{4}-\d{2}-\d{2}$")`
Shape of a CUSIP	`cusips.str.fullmatch(r'[0-9A-Z]{9}')`	200 of 200. 4 damaged copies fail.	`=REGEXTEST(B2, "^[0-9A-Z]{9}$")`
A description	`re.fullmatch(description_pattern, text)`	8 of 8, as numbers and dates	`=TEXTSPLIT(TRIM(A2), " ")  =VALUE(C2)`

TEXTBEFORE, TEXTAFTER, TEXTSPLIT: Microsoft 365 and Excel 2024. REGEXTEST, and =REGEXEXTRACT(A2, pattern, 2) for the groups: Excel for Microsoft 365.

Every version of Excel: =LEN(B2)=9 and =EXACT(B2, UPPER(B2)). The eight descriptions and the 200 CUSIPs are invented, from the synthetic holdings file.

Cleaning text in steps

One sentence of the statement dated 2023-11-01, through the four steps of clean()

	Python	Excel, with the sentence in A2
As published: 23 pieces		
`In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.`		
Step 1: lower case	`step1 = sentence.lower()`	`=LOWER(A2)`
`in support of these goals, the committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.`		
Step 2: non-letters to spaces	`step2 = re.sub(r'[^a-z\s]', ' ', step1)`	`=REGEXREPLACE(LOWER(A2), "[^a-z\s]", " ")`
`in support of these goals  the committee decided to maintain the target range for the federal funds rate at  to  percent`		
Step 3: one space between words	`step3 = re.sub(r'\s+', ' ', step2).strip()`	`=TRIM(B2)`
`in support of these goals the committee decided to maintain the target range for the federal funds rate at to percent`		
Step 4: split into 21 tokens	`tokens = step3.split(' ')`	`=TEXTSPLIT(C2, " ")`

`in` `support` `of` `these` `goals` `the` `committee` `decided` `to` `maintain` `the` `target` `range` `for` `the` `federal` `funds` `rate` `at` `to` `percent`

```
def clean(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text.split(' ')
```

Dropping the digits is a decision

The job is to count words, and 5, 1 and 4 are not words. The figures were taken from the text as published, before any cleaning.

Whole statement: 344 pieces by split(), 343 tokens.

Without REGEXREPLACE: one SUBSTITUTE for each character, nested.

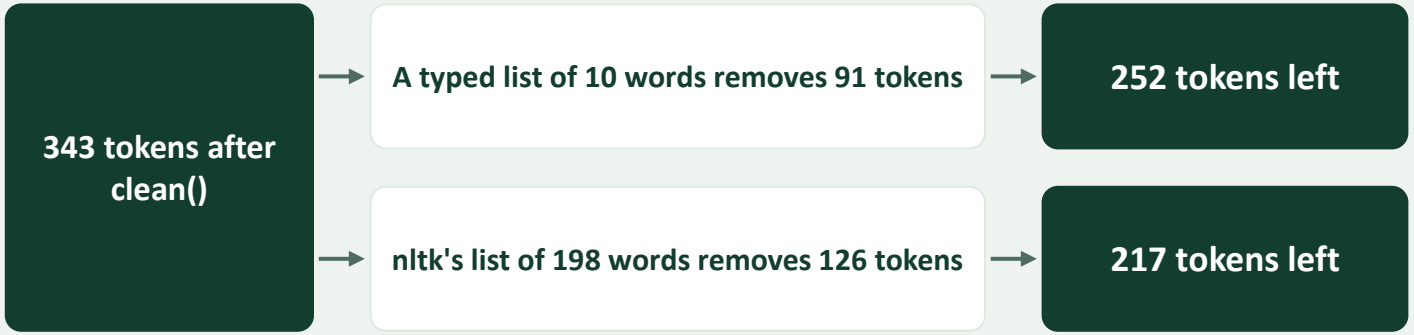
Stopwords: words every English text has

Most common tokens, statement dated 2023-11-01

All tokens	Count	After the nltk list	Count
the	29	committee	11
and	16	inflation	9
to	12	policy	7
of	11	monetary	6
committee	11	economic	5
inflation	9	percent	4

The words at the top of the first list are in any English text. They are called stopwords, and for counting they are usually removed.

The second list is a count and nothing more.



Different tokens: 180 before, 152 after the nltk list.

The single letter s, left over from Committee's, is on the nltk list.

```
short_list = ['the', 'of', 'to', 'and', 'a', 'in', 'is', 'that', 'for', 'its']
stop_words = stopwords.words('english')   # 198 words
kept = [token for token in statement_tokens if token not in stop_words]
Counter(kept).most_common(10)
```

The same in Excel: the 343 tokens in B2:B344, the stopword list in column A of a sheet named stop

`=COUNTIF(B2:B344, "the")`	returns 29	`=SUMPRODUCT(--(COUNTIF(stop!A:A, B2:B344) = 0))`	returns 217
`=COUNTIF(stop!A:A, B2) > 0`	TRUE or FALSE, filled down	`=FILTER(B2:B344, COUNTIF(stop!A:A, B2:B344) = 0)`	217 tokens

FILTER is in Excel 2021 and Microsoft 365. The SUMPRODUCT count works in every version. A pivot table of the tokens gives every count.

Stemming and its limits

Words in the statements	Stem	What happened
remain, remained, remains	remain	forms of one word, together
assess, assessing, assessments	assess	forms of one word, together
reserve, reserves	reserv	forms of one word, together
inflation	inflat	a stem is not a word
policy	polici	a stem is not a word
committee	committe	a stem is not a word
economic	econom	kept apart from economy
economy	economi	kept apart from economic
production, productivity	product	two words, merged
time, timing	time	two words, merged

The two limits: stemming can keep related words apart, and it can merge words that are not the same. A stem count is rougher than a word count. Label it as a stem count.

The Porter stemmer is a fixed list of rules about English endings. It needs no download.

152 to 146

different tokens become different stems

5

times for the stem remain: remains had 3

```
stemmer = PorterStemmer()
stems = [stemmer.stem(token) for token in kept]
```

In Excel

Excel has no function for stemming. The nearest hand tool is a wildcard.

```
=COUNTIF(B2:B344, "remain*")
```

returns 5: every token that starts with remain

Text into numbers: one line, 46 statements

A column of strings is a Series, and its string methods sit behind .str

Job	pandas, on a column of text	Excel, with the 46 texts in C2:C47	Result
Lower case	`.str.lower()`	`=LOWER(C2), filled down`	46 texts
Characters in each text	`.str.len()`	`=LEN(C2), filled down`	2,300 for 2023-11-01
Contains a piece of text	`.str.contains('balance').sum()`	`=SUMPRODUCT(--ISNUMBER(SEARCH("balance", C2:C47)))`	29 statements
Times a piece appears	`.str.count('inflation')`	see the formula under the table	10 for 2021-01-27
A function on every row	`.apply(clean)`	a formula filled down a column	16,446 tokens

`=(LEN(C2) - LEN(SUBSTITUTE(LOWER(C2), "inflation", ""))) / LEN("inflation")` returns 10 for the statement dated 2021-01-27

Whole word: .str.contains(r'\bbalance\b') is True for 25 of the 46 statements. The letters are in 29: imbalances and balances hold them.

```
statements['tokens'] = statements['text'].apply(clean)
all_counts = Counter()
for tokens in statements['kept']:
    all_counts.update(tokens)
```

16,446 tokens in the 46 statements.

10,204 after stopword removal. 507 different tokens.

Most common: committee 477, inflation 324, policy 196.

Test a pattern by a count: 46 statements, 41 found

The range pattern matched in 41 of the 46 statements, and not in 5.

In those 5 the hyphen is the non-breaking hyphen, written \u2011.

`range_pattern = r'[\d/\u2011-]+ to [\d/\u2011-]+ percent'`

With either hyphen in the class, all 46 statements have a range.

`=UNICODE(MID(C2, n, 1))` 45 plain, 8209 this one (file as UTF-8)

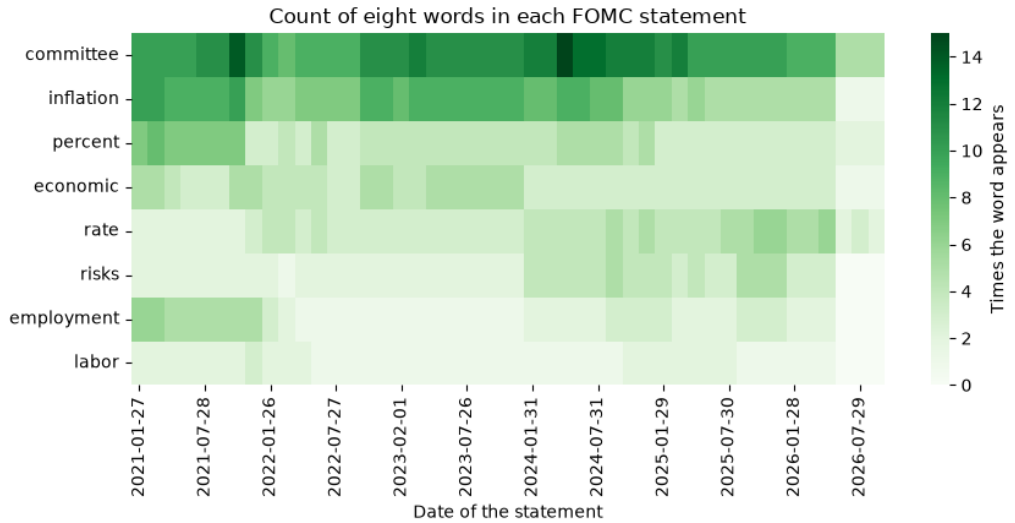
`=SUBSTITUTE(C2, UNICHAR(8209), "-")` replaces one with the other

The document-term table and its heatmap

One row for each statement, one column for each word, and each cell a count

Statement dated	committee	inflation	percent	economic	rate	risks	employment	labor
2021-01-27	10	10	7	5	2	2	6	2
2021-03-17	10	10	8	5	2	2	6	2
All 46 statements	477	324	190	163	163	121	106	62

```
counters = statements['kept'].apply(Counter)
term_table = pd.DataFrame({term:
    [counter[term] for counter in counters]
    for term in terms})
sns.heatmap(term_table.T, cmap='Greens', xticklabels=4,
    cbar_kws={'label': 'Times the word appears'},
ax=ax)
```



Mean count per 100 tokens kept, by year

Year	committee	inflation
2021	3.47	2.93
2022	4.17	3.23
2023	5.18	4.13
2024	5.94	3.64
2025	5.05	2.57
2026	5.31	1.91

A count depends on how long the document is. Per 100 tokens kept puts a long statement and a short one on the same footing.

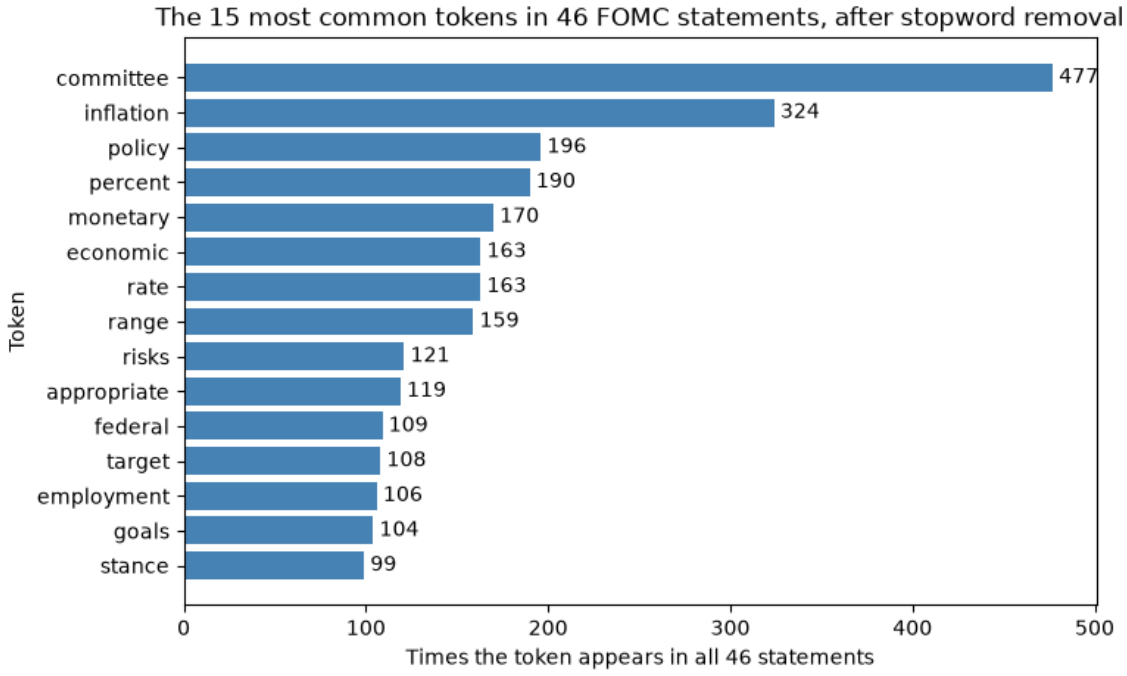
The committee row holds counts from 5 to 15, the labor row from 0 to 3.

```
per_100 = term_table.div(statements['n_kept'].values, axis=0) * 100
```

In Excel: the counts, with Home, Conditional Formatting, Color Scales.

The most common tokens and a short vocabulary

The 15 most common tokens after stopword removal



How much a short vocabulary keeps

Tokens kept	Share of the 10,204 kept tokens
10 most common	20.4 percent
20 most common	30.0 percent
50 most common	48.0 percent
All 507 different tokens	100 percent

```
full_table = pd.DataFrame(list(counters)).fillna(0).astype(int)
top_tokens = full_table.sum().nlargest(50).index
top_table = full_table[top_tokens]   # 46 rows, 50 columns
```

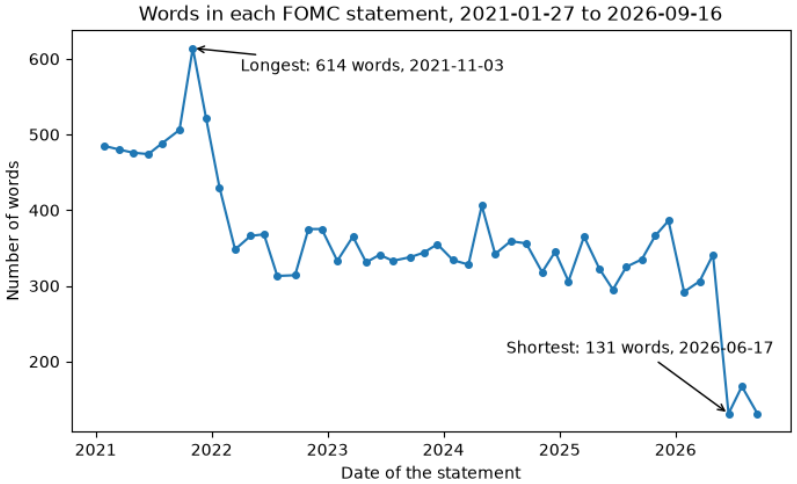
70 of the 507 tokens appear only once in the whole file.

scikit-learn's CountVectorizer(max_features=50) builds this in one step, with its own rules for what a word is.

In Excel: a pivot table of statement, token, and count.

Word count and shared wording over time

Words in each statement



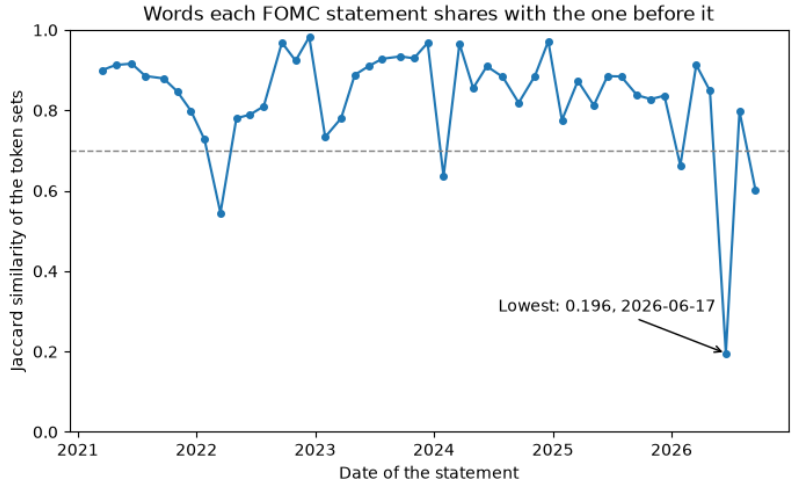
Mean 359.3 words, median 344.5.

Longest: 614 words, the statement dated 2021-11-03.

Shortest: 131 words, the statement dated 2026-06-17.

```
statements['words'] = statements['text'].str.split().str.len()
```

Words each statement shares with the one before it



`similarity = tokens in both / tokens in either`

Mean 0.832. Highest 0.982, the statement dated 2022-12-14.

Lowest 0.196, the statement dated 2026-06-17. Five are below 0.7.

```
return len(first_set & second_set) / len(first_set | second_set)
```

The charts show how long each statement is and how much wording changed. They do not say why, or what any wording means.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Measure, search, slice, split, join and replace text with string methods
- Write and test a regular expression: figures, bond descriptions, CUSIPs
- Clean text in steps, remove stopwords, and stem, knowing the limits
- Count tokens with Counter, and work on a column with .str and .apply()
- Build a document-term table, draw it, and compare two documents

Week 4, Day 3 Case Study: Statement Sentiment